Naturally, this trick also works via `ReflectionMethod::invoke()`.

This has been just a brief taste of PHP's reflection API. You are most likely to encounter it buried inside frameworks and powering the kinds of libraries that focus on testing and code analysis, and that's a little beyond the scope of this book. If you're interested in learning more, you can start with the official documentation at https://www.php.net/manual/en/book.reflection.php. There is, however, a new (for PHP 8) feature that makes particular use of reflection, one that you might make good use of generally in your own code.

Attributes

PHP 8 introduces a new way to provide information about elements of your code (such as classes, methods, functions) without altering that code, in the form of *attributes*. You use classes and (especially) interfaces to define structure and implementation, and you can add information to them to control other aspects of your software, but this comes with a price: you can end up with interfaces bloated with public constants, properties, and/or methods. Attributes allow you to annotate your code with metadata, which is accessed via reflection, keeping your runtime interfaces trim and focused.

Attributes are – at heart – classes in their own right, which are attached to the constructs in your code using a special syntax `#[AttributeName(param: val, ...)]`, which goes immediately before the statement they are intended to mark. There are no PSR standards for attribute syntax (PSR-12 predates the addition of attributes) but the convention tends to be that an attribute statement starts on its own line, and follows the PSR-12 standards for class instantiation. PHP provides several predefined attributes and a mechanism for creating your own, and it's the reflection API that powers them at runtime. Here's an example of how to use attributes using the predefined `#[Deprecated]`:

```
#[Deprecated(message: 'Time to migrate!', since: '1.0')]
function oldFunc(): string {
    return 'Use something else' . PHP_EOL;
}

print oldFunc();
```